

万字详解！AI Harness 入门与实践！一篇博文给你讲透！

Harness Agent 的基本概念

1. 先说结论：AI 应用开发正在从“会问 AI”变成“会给 AI 搭工作台”



🔥 过去一年，很多人做 AI 应用开发的姿势是这样的：

“帮我写一个登录页。”

“帮我修一下这个 bug。”

“帮我生成一个 Python 爬虫。”

“帮我把这个函数改成异步。”

这类做法通常被叫作 **Vibe Coding**。大白话说，就是你把想法说出来，让 AI 顺着你的感觉往前写代码。你不一定逐行看代码，也不一定一开始就把需求写得特别严谨。你更像是在和一个很快的程序员聊天：你说一个大概方向，它先做一个版本；你试一下，不满意，再让它改。

Vibe Coding 的价值很明显：快。以前一个小工具可能要半天，现在十几分钟就能跑起来。以前你不懂前端、不懂后端、不懂脚本，也许做不出来，现在你可以靠自然语言先做一个可用版本。

但问题也很明显：快，不等于稳。能跑，不等于能上线。看起来能用，不等于边界条件正确。AI 写出来的代码，有时候像临时搭的棚子：遮雨可以，但不一定经得起长期使用。

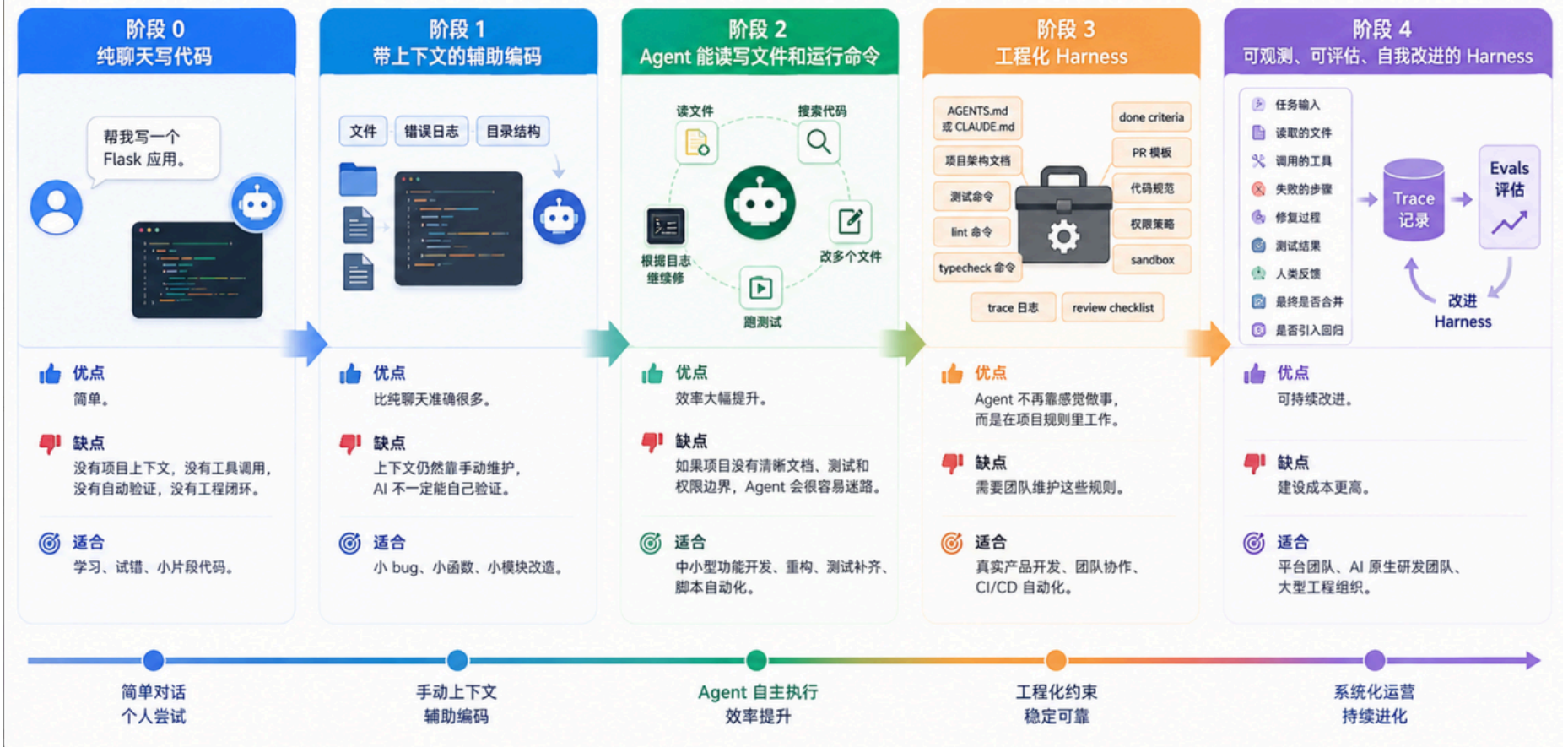
所以现在真正值得关注的变化，不是“AI 会不会写代码”，而是：

我们怎么让 AI 在一个可验证、可约束、可回滚、可观察的环境里写代码？

这就是本文要讲的核心：**Harness Engineering**。

@xdbc
lass

AI 应用开发的成熟度模型：从简单对话到可观测、自我改进的工程系统



你可以先把它理解成一句话：

Harness Engineering 不是教你写更玄的提示词，而是教你给 AI Agent 搭一个能干活、能自查、能留痕、能被人类接管的工程环境。

如果 Vibe Coding 是“让 AI 凭感觉开车”，那 Harness Engineering 就是“给 AI 修路、画线、装红绿灯、加仪表盘、设限速、放安全员”。

1. Harness 到底是什么意思？

Harness 到底是什么意思？

Harness 是包在 AI 模型外面的一层运行系统，是模型和真实工程环境之间的“工作底座”。



总结： 同一个模型，在普通聊天窗口里表现一般，但在有 Harness 的工程环境中，能力会被极大释放。不是模型换了大脑，而是它有了**更好的工作台**。

🔥 “Harness”这个词本来不是 AI 专有词。它在英文里有“马具、装备、把力量约束并利用起来”的意思。放到软件工程里，我们经常听过一个词：**test harness**，测试夹具、测试套件、测试环境。

它的意思是：你不是只写一个函数，然后靠肉眼看结果；你要给这个函数配一套输入、输出、断言、运行环境，用这套环境来判断它到底有没有做好。

到了 AI Agent 时代，Harness 的意思被扩展了。

一个 AI Coding Agent 不是单纯的模型。模型只是大脑。真正让它能干活的，是它周围的一整套东西：

- 它能看到哪些文件；
- 它能调用哪些工具；
- 它能不能跑测试；
- 它能不能改文件；
- 它能不能访问网络；
- 它改完以后怎么判断自己完成了；
- 它失败以后怎么定位原因；
- 它每一步操作有没有日志；
- 它什么时候必须停下来问人；
- 它的上下文怎么组织；
- 它的长期项目记忆放在哪里；
- 它产生的变更怎么被审查和合并。

这些东西加起来，就是一个 Agent 的 **Harness**。

所以，**Harness** 不是单个提示词，也不是单个工具，而是模型和真实工程环境之间的“工作底座”。

更准确一点说：

Harness 是包在 AI 模型外面的一层运行系统。它负责把用户目标变成可执行任务，把项目上下文喂给模型，把模型输出转成工具调用，把工具结果反馈给模型，并用测试、权限、日志、审查机制判断任务是否真的完成。

这也是为什么同一个模型，在普通聊天窗口里表现一般，但放到 Codex、Claude Code、Cursor、Windsurf 这类工程环境里，会显得强很多。不是模型突然换了脑子，而是它有了更好的工作台。

2. Harness Agent 是什么？

什么是 Harness Engineering?

从“说清楚”到“给够资料”再到“搭好环境”，让 AI 在工程闭环中持续、可靠、可审查地完成复杂任务。



“Harness Agent”这个词有两层意思，需要分开讲。

第一层，是泛化意义上的 **harnessed agent**，也就是“被 Harness 包起来的 Agent”。比如一个 coding agent，它有如下能力：

- 读项目文件；
- 搜索代码；
- 修改代码；
- 运行 shell 命令；
- 跑测试；
- 读取失败日志；
- 继续修复；
- 生成变更说明；
- 请求人类批准高风险操作。

这类 Agent 的重点不是“它能聊天”，而是“它能在工具环境里循环执行任务”。这就已经不是传统 chatbot 了，而是一个带工作流的执行体。

第二层，是特定产品里的 **Harness Agent**。比如 Harness.io 这家公司提供的 Harness 平台里，有 Worker Agents、Agent Marketplace、Harness MCP Server 等概念。它们面向 DevOps、CI/CD、测试、安全、成本优化等场景，可以让用户创建、配置和运行工作型 Agent。你可以把它理解成“把智能体放进软件交付流水线里”，例如 PR review、pipeline failure summarizer、IaC plan safety、zero-day remediation 等。

两者关系可以这样理解：

- 泛化意义上的 harness agent：任何被工作底座包起来、能调用工具做事的 AI Agent。
- Harness.io 里的 Worker Agent：一个具体平台产品里的 Agent 能力，主要服务软件交付、DevOps、安全、测试等流程。

本文主要讲第一种，也就是更通用的 AI 应用开发方法论；同时会穿插 Harness.io 这种具体平台如何落地。

3. 什么是 Harness Engineering?

如果 Harness 是工作底座，那么 Harness Engineering 就是“设计、实现、维护这个底座”的工程方法。

过去我们讲 Prompt Engineering，核心问题是：

｜ 我应该怎么问 AI，AI 才能回答得更好？

后来我们讲 Context Engineering，核心问题变成：

｜ 我应该给 AI 什么上下文，AI 才能在正确的信息里工作？

而 Harness Engineering 又往前走了一步：

｜ 我应该给 AI 什么样的任务环境、工具边界、验证机制、状态管理和反馈循环，AI 才能持续、可靠、可审查地完成复杂任务？

这三者的层次不同。

3.1 Prompt Engineering：把话说清楚

比如：

代码块

- ```
1 你是一个资深 Python 工程师。请帮我实现一个订单折扣函数，要求：
2 1. 满 100 减 10；
3 2. VIP 用户额外 95 折；
4 3. 返回值保留两位小数。
```

这已经比“帮我写个折扣函数”好多了。

但它仍然主要依赖模型一次性生成正确答案。

### 3.2 Context Engineering：把资料给够

比如你告诉 AI：

代码块

- ```
1  这是我们项目的目录结构：
2  src/
3  pricing/
4      discount.py
5  tests/
6      test_discount.py
7
8  这是现有代码：
9  ...
10
11  这是我们的编码规范：
12  - 金额统一使用 Decimal；
13  - 不允许用 float 处理价格；
14  - 测试必须覆盖普通用户、VIP 用户、边界值。
```

这样 AI 不再是凭空写代码，而是基于项目真实上下文来写。

3.3 Harness Engineering：把工作环境搭好

再进一步，你不仅给它资料，还给它一套可以执行的流程：

代码块

- ```
1 你可以：
2 1. 读取 src/ 和 tests/ 下的文件；
3 2. 修改业务代码和测试代码；
4 3. 运行 pytest；
5 4. 如果测试失败，读取失败日志并继续修；
```

```
6 5. 最终必须输出：
7 - 修改了哪些文件；
8 - 为什么这么改；
9 - pytest 是否通过；
10 - 是否存在未覆盖风险。
11 你不可以：
12 1. 删除数据库；
13 2. 访问外部网络；
14 3. 修改 pyproject.toml 以外的构建配置；
15 4. 在测试没通过时声称完成。
```

这就是 Harness Engineering 的味道。

它不是只靠“说得更好”，而是靠：

- 工具；
- 权限；
- 反馈；
- 测试；
- 日志；
- 状态；
- 结构化输出；
- 人类审批。

让 AI 进入一个工程闭环。

## 4. 为什么 Vibe Coding 不够？

Vibe Coding 很适合从 0 到 1。比如你要做：

- 一个内部小工具；
- 一个演示 Demo；
- 一个一次性脚本；
- 一个低风险页面；
- 一个探索性原型；
- 一个产品想法验证。

你用自然语言和 AI 对话，很快就能看到东西。这种体验过去没有，所以它很迷人。

但当项目开始变复杂，Vibe Coding 会遇到几个硬问题。

### 4.1 需求会漂

你一开始说“做个记账工具”，AI 可能会给你做一个很漂亮的页面。你继续说“加个分类统计”，它又加了图表。你再说“支持多人协作”，它可能开始乱改数据结构。到最后，你自己都不清楚这个项目原来的核心目标是什么。

这不是 AI 特有的问题。人类团队也会这样。区别是 AI 写得太快，漂得也太快。

### 4.2 上下文会爆

AI Agent 做复杂任务时，会读很多文件、跑很多命令、产生很多日志、经历多轮对话。上下文窗口再大，也不是无限的。上下文越乱，模型越容易忘记早期约束，或者把过期信息当成最新事实。

所以工程里不能把所有东西都塞进一个大提示词。要有文档地图，要有短入口，要有可检索的项目知识库，要有“系统事实来源”。

### 4.3 代码能跑，但不一定对

一个函数在某个样例下能跑，不代表它满足所有业务边界。

比如金额计算，最怕这些问题：

- float 精度误差；
- 四舍五入规则错误；
- 折扣顺序错误；
- 边界值 99.99 和 100.00 处理错误；
- 负数金额没有拒绝；
- 空订单没有处理；
- VIP 和普通用户逻辑混在一起。

如果没有测试，AI 很容易写出“看起来对”的代码。

### 4.4 AI 很会“自信地完成”

很多 Agent 会在没有真正验证时说“已完成”。这对工程来说非常危险。人类工程师至少知道自己有没有跑测试，但 Agent 如果没有被要求跑测试，或者没有权限跑测试，它可能只靠代码表面判断。

Harness Engineering 的关键，就是不要让“模型觉得完成了”成为完成标准。

真正的完成标准应该是：

- 测试通过；
- lint 通过；
- typecheck 通过；
- 构建通过；
- 关键路径验证通过；
- 人类审查通过；
- 变更说明清楚；
- 可回滚。

### 4.5 安全边界不清楚

如果你给 Agent 一个 shell，它就可能运行命令。如果它能运行命令，就可能读文件、删文件、发请求、安装包、修改配置。不是说 Agent 一定会作恶，而是复杂任务里总会有误操作。

所以 Harness 必须有权限边界：

- 哪些文件可读；
- 哪些文件可写；
- 哪些命令可执行；
- 是否允许联网；
- 是否允许访问密钥；
- 是否允许执行部署；
- 哪些操作必须人类批准。

没有边界的 Agent，不适合进入真实工程。

---

## 5. 从 Vibe Coding 到 Harness Engineering 的五阶段



# Vibe Coding 的 5 大硬问题与 Harness Engineering 解决思路

@小滴技术

当项目变复杂，Vibe Coding 会遇到工程级挑战，需要 Harness Engineering 保驾护航。

1 需求会漂

目标不断偏移，核心价值模糊

做个记账工具

加个分类统计

支持多人协作

???

需求持续变更，方向越来越模糊

AI 写得又快，漂得也很快

2 上下文会爆

信息过载，模型遗忘关键约束

文件

命令

日志

对话

上下文窗口有限

信息过期被误用

文件、日志、对话堆积如山

模型容易忘记早期约束或使用过期信息

3 代码能跑，但不一定对

样例通过 ≠ 业务正确

```
function calculate(order) {
 return order.amount * 0.9
}
```

float 精度误差

四舍五入规则错误

折扣顺序错误

边界值处理错误

负数金额未拒绝

空订单未处理

VIP/普通逻辑混乱

没有测试，很容易写出“看起来对”的代码

4 AI 很会“自信地完成”

未验证，却报告“已完成”

“已完成”  
所有功能已实现

未运行测试

未构建验证

未人工审查

不要让“模型觉得完成了”成为完成标准

5 安全边界不清楚

误操作风险高，真实工程不可控

读文件

删文件

发请求

安装包

改配置

执行命令

没有边界的 Agent，不适合进入真实工程

真正的完成标准

测试通过

lint 通过

typecheck 通过

构建通过

关键路径验证通过

人类审查通过

变更说明清楚

可回滚

Harness Engineering 的核心:

用工程化手段，让 AI 可靠、可控、可交付。

我们可以把 AI 应用开发的成熟度分成五个阶段。

## 阶段 0：纯聊天写代码

你打开一个聊天窗口，说：

代码块

1 帮我写一个 Flask 应用。

AI 给你一段代码。你复制到本地跑。

优点：简单。

缺点：没有项目上下文，没有工具调用，没有自动验证，没有工程闭环。

适合：学习、试错、小片段代码。

## 阶段 1：带上下文的辅助编码

你把文件、错误日志、目录结构贴给 AI，让它帮你改。

优点：比纯聊天准确很多。

缺点：上下文仍然靠手动维护，AI 不一定能自己验证。

适合：小 bug、小函数、小模块改造。

## 阶段 2：Agent 能读写文件和运行命令

你使用 Codex、Claude Code、Cursor Agent、Windsurf Agent 等工具，让 Agent 直接在项目里工作。

它可以：

- 读文件；
- 搜索代码；
- 改多个文件；
- 跑测试；



- 根据日志继续修。

优点：效率大幅提升。

缺点：如果项目没有清晰文档、测试和权限边界，Agent 会很容易迷路。

适合：中小型功能开发、重构、测试补齐、脚本自动化。

### 阶段 3：工程化 Harness

你开始给 Agent 明确的工作台：

- AGENTS.md 或 CLAUDE.md ；
- 项目架构文档；
- 测试命令；
- lint 命令；
- typecheck 命令；
- done criteria；
- PR 模板；
- 代码规范；
- 权限策略；
- sandbox；
- trace 日志；
- review checklist。

优点：Agent 不再靠感觉做事，而是在项目规则里工作。

缺点：需要团队维护这些规则。

适合：真实产品开发、团队协作、CI/CD 自动化。

### 阶段 4：可观测、可评估、自我改进的 Harness

你进一步把 Agent 的每次运行都记录下来：

- 任务输入；
- 读取了哪些文件；
- 调用了哪些工具；
- 哪一步失败；
- 怎样修复；
- 测试结果；
- 人类反馈；
- 最终是否合并；
- 是否引入回归。

然后把这些 trace 转成 evals，反过来改进 Harness。

这时你不只是“用 Agent 写代码”，你是在运营一个“Agent 工程系统”。

优点：可持续改进。

缺点：建设成本更高。

适合：平台团队、AI 原生研发团队、大型工程组织。

---

## 6. Harness Engineering 的核心组件

# Harness Engineering 的核心组件

@小滴课堂

构建可靠、可控、可扩展的 AI Agent 工程底座

## 1 任务规格：让目标别漂

```
Task Spec
Goal: 实现订单折扣计算
Non-goals: 不做优惠券系统,
 不接入支付, 不修改数据库结构
Inputs:
- items: 商品列表
- user_type: normal 或 vip
Rules:
- 满 100 减 10; VIP 额外 95 折;
- 金额使用 Decimal; 保留两位小数;
- 负数价格必须报错。
Done When: pytest 全部通过;
 新增边界测试;
 不引入 float;
 输出变更说明。
```



## 2 上下文选择：少而准，按需读取

```
project/
├── AGENTS.md
├── docs/
│ ├── architecture.md
│ ├── pricing-rules.md
│ └── testing.md
└── ...
```

**AGENTS.md (地图)**

- ✓ 项目简介
- ✓ 先读这些 (地图)
- ✓ 常用命令
- ✓ 完成标准

详细内容放在 docs/  
按需读取，短、准、可验证

## 3 工具访问：少而精，可组合

read\_file

write\_file

search\_code

run\_tests

run\_lint

git\_diff

create\_pr

像 Unix 命令，稳定、可组合

避免 200+ 业务 API，降低选择错误

## 4 项目记忆：沉淀到仓库，留下路标

```
docs/
├── architecture.md
├── decisions/
│ ├── 0001-use-decimal-for-money.md
│ └── 0002-discount-order.md
├── testing.md
├── known-issues.md
├── tech-debt.md
└── ...
```

重要事实写到仓库，  
对话会结束，文档会留下。  
下一次进来，  
不用从头问。

## 5 状态管理：复杂任务需要进度文件

```
exec-plans/2028-05-pricing-refactor.md
Goal: 重构 pricing 模块，统一 Decimal 金额计算。
Milestones:
- [x] 梳理现有接口
- [x] 增加边界测试
- [] 重构 float 计算
- [] 单元测试
- [] 更新文档
Current Status: 已发现 cart.py 和 discount.py
中存在 float 计算。
Risks: 订单导出 CSV 可能精度丢失。
```

- 明确目标
- 分解里程碑
- 记录现状
- 识别风险
- 可恢复、可继续

## 6 验证机制：不要相信“我觉得对”

单元测试

集成测试

lint

typecheck

build

snapshot

golden file

API contract test

数据迁移 dry run

回归测试

最有价值的反馈：清晰、可定位、可修复

FAILED tests/test\_discount.py::test\_vip\_discount  
Expected Decimal("85.50"), got Decimal("90.25")

## 7 权限和沙箱：分級授权，在围栏里跑

**Level 0**  
只读

- 读文件
- 阅读代码
- 不改文件
- 不执行命令

**Level 1**  
本地跑测

- 读文件区文件
- 跑测试 / lint
- 不联网
- 不访问远程

**Level 2**  
受控外网访问

- 访问指定 MCP 工具
- 高风险操作
- 需确认

**Level 3**  
CI/CD 自动化

- 创建 PR
- 触发 CI
- 不直接部署
- 生产

**Level 4**  
存库位跑

- 自动修复
- 开 PR
- 请求 review
- 生产变更需人类批准

循序渐进，先在低风险环境证明自己

## 8 可观测性：每一步都要能复盘

```
{
 "task_id": "pricing-discount-001",
 "started_at": "2028-05-28T10:00:00Z",
 "steps": [
 {
 "type": "read_file",
 "path": "src/cart.py",
 "status": "success"
 },
 {
 "type": "write_file",
 "path": "src/cart.py",
 "status": "success"
 },
 {
 "type": "run_tests",
 "command": "pytest",
 "exit_code": 1,
 "summary": "test_vip_discount failed",
 "type": "write_file",
 "path": "src/cart.py",
 "status": "success"
 },
 {
 "type": "run_tests",
 "command": "pytest",
 "exit_code": 0
 }
],
 "final_status": "passed"
}
```

- 记录关键步骤
- 保留上下文
- 定位问题
- 评估质量
- 建立信任

## 9 人类接管：读自动的自动，该停的停

- ✓ 改测试：自动
- ✓ 跑 lint：自动
- ✓ 重构工具函数：自动
- ✗ 修改核心支付逻辑：需要 review
- ✗ 删除数据：必须拒绝
- ✗ 改 CI/CD 权限：必须审批
- ✗ 访问生产密钥：不允许

目标不是让人消失，  
而是让人从  
“弄巧每一行代码”  
变成  
“设计规则、审查结果，  
处理关键决策”。

核心目标：让 AI Agent 在明确的规则、可验证的环境、可观测的流程中，稳定高效地完成工程任务

可预测 可验证 可控 可迭代 可信

一个成熟的 Harness，至少应该包含下面这些模块。

## 6.1 任务规格：让目标别漂

任务规格不是一句“做个功能”。它应该包括：

代码块

```
1 # Task Spec
2
3 ## Goal
4 实现订单折扣计算。
5
6 ## Non-goals
7 不做优惠券系统。
8 不接入支付。
9 不修改数据库结构。
10
11 ## Inputs
12 - items: 商品列表
13 - user_type: normal 或 vip
14
15 ## Rules
16 - 满 100 减 10;
17 - VIP 在满减后额外 95 折;
18 - 金额使用 Decimal;
19 - 返回保留两位小数;
20 - 负数价格必须报错。
21
22 ## Done When
23 - pytest 全部通过;
24 - 新增边界测试;
25 - 不引入 float;
26 - 输出变更说明。
```

这类规格很朴素，但非常有用。它把“感觉”变成“合同”。

## 6.2 上下文选择：不要把所有东西都塞给 AI

很多人以为上下文越多越好。实际不是。上下文是稀缺资源。你给太多，模型会抓不住重点。

更好的做法是：

- 根目录放一个短的 `AGENTS.md` ；
- `AGENTS.md` 只做地图；
- 详细内容放到 `docs/` ；
- 让 Agent 按需读取；
- 每个文档有明确用途；
- 文档保持短、准、可验证。

比如：

代码块

```
1 # AGENTS.md
2
3 ## 项目简介
4 这是一个订单和价格计算服务。
5
6 ## 先读这些
7 - docs/architecture.md: 模块边界
8 - docs/pricing-rules.md: 价格规则
9 - docs/testing.md: 测试命令和测试策略
10
11 ## 常用命令
12 - pytest
13 - ruff check .
14 - mypy src
15
16 ## 完成标准
17 任何改动都必须通过 pytest。
18 涉及金额计算时必须新增边界测试。
```

这比一个几千行的大说明靠谱得多。

## 6.3 工具访问：工具少而精，比工具多更重要

Agent 最怕什么？不是没有工具，而是工具太多、接口太碎、语义太复杂。

一个好的 Harness 应该给 Agent 提供少量、稳定、可组合的工具。例如：

- `read_file(path)`
- `write_file(path, content)`
- `search_code(query)`
- `run_tests(command)`
- `run_lint(command)`
- `git_diff()`
- `create_pr(title, body)`

工具越像 Unix 命令，越容易组合。工具越像“200 个业务 API 摆在面前”，Agent 越容易选错。

## 6.4 项目记忆：把重要事实写到仓库，而不是留在聊天里

Agent 的对话会结束，上下文会压缩，记忆会丢。但仓库里的文件会留下来。

所以重要事实应该沉淀成文件：



代码块

```
1 docs/
2 architecture.md
3 decisions/
4 0001-use-decimal-for-money.md
5 0002-discount-order.md
6 testing.md
7 known-issues.md
8 tech-debt.md
```

这就像给 Agent 留路标。下一次它进来，不用从头问。

## 6.5 状态管理：复杂任务必须有进度文件

长任务不能只靠对话记录。最好让 Agent 写一个计划文件：

代码块

```
1 # exec-plans/2026-05-pricing-refactor.md
2
3 ## Goal
4 重构 pricing 模块，统一 Decimal 金额计算。
5
6 ## Milestones
7 - [x] 梳理现有价格入口
8 - [x] 增加边界测试
9 - [] 替换 float 计算
10 - [] 跑全量测试
11 - [] 更新文档
12
13 ## Current Status
14 已发现 cart.py 和 discount.py 中存在 float 计算。
15
16 ## Risks
17 - 订单导出 CSV 可能依赖旧格式。
```

这样就算上下文被压缩，Agent 也能回到这个文件继续工作。

## 6.6 验证机制：不要相信“我觉得对”

验证机制是 Harness 的灵魂。

至少要有：

- 单元测试；
- 集成测试；
- lint；
- typecheck；
- build；
- snapshot；
- golden file；
- API contract test；
- 数据迁移 dry run；
- 回归测试。

对 Agent 来说，最有价值的反馈不是“你写得不错”，而是：

代码块

```
1 FAILED tests/test_discount.py::test_vip_discount
2 Expected Decimal("85.50"), got Decimal("90.25")
```

这种反馈非常清楚。Agent 能基于它继续修。

### 6.7 权限和沙箱：让 Agent 在围栏里跑

Agent 需要自由，但不能无限自由。

一个实际项目里可以分几层权限：

代码块

```
1 Level 0：只读
2 - 允许读文件、搜索代码
3 - 不允许改文件、不允许执行命令
4
5 Level 1：本地修改
6 - 允许改工作区文件
7 - 允许跑测试和 lint
8 - 不允许联网
9 - 不允许访问密钥
10
11 Level 2：受控外部访问
12 - 允许访问指定 MCP 工具
13 - 高风险操作需要确认
14
15 Level 3：CI/CD 自动化
16 - 允许创建 PR
17 - 允许触发 CI
18 - 不允许直接部署生产
19
20 Level 4：有限自治
21 - 在明确策略下自动修复、开 PR、请求 review
22 - 生产变更仍需人类批准
```

不要一上来就给 Agent 生产权限。先让它在低风险环境里证明自己。

### 6.8 可观测性：每一步都要能复盘

Agent 失败不可怕。可怕的是失败以后你不知道它为什么失败。

所以 Harness 应该记录：

代码块

```
1 {
2 "task_id": "pricing-discount-001",
3 "started_at": "2026-05-28T10:00:00Z",
4 "steps": [
5 {
6 "type": "read_file",
7 "path": "src/cart.py"
8 },
9 {
10 "type": "write_file",
11 "path": "src/cart.py"
12 },
13 {
14 "type": "run_tests",
```

```
15 "command": "pytest",
16 "exit_code": 1,
17 "summary": "test_vip_discount failed"
18 },
19 {
20 "type": "write_file",
21 "path": "src/cart.py"
22 },
23 {
24 "type": "run_tests",
25 "command": "pytest",
26 "exit_code": 0
27 }
28],
29 "final_status": "passed"
30 }
```

这类 trace 对调试、评估和团队信任都很重要。

### 6.9 人类接管：不是全自动才高级

很多人把 AI Agent 想成“全自动”。但真实工程里，高级不是全自动，而是该自动的自动，该停的停。

比如：

- 改测试：自动；
- 跑 lint：自动；
- 修改核心支付逻辑：需要 review；
- 删除数据：必须拒绝；
- 改 CI/CD 权限：必须审批；
- 访问生产密钥：不允许。

Harness Engineering 的目标不是让人消失，而是让人从“手写每一行代码”变成“设计规则、审查结果、处理关键决策”。

## 7. 一个可以验证的最小 Harness 案例

下面这个案例不依赖真实 LLM。它用一个“假 Agent 提案”来模拟 AI 写补丁，然后用 Harness 来决定是否接受补丁。

为什么不用真实模型？因为我们要让案例可验证。真实模型输出每次可能不同，不适合做入门示范。这个例子重点是 Harness 思路：**不是谁写了代码重要，而是代码必须经过验证才算完成。**

### 7.1 项目结构

代码块

```
1 mini-harness-demo/
2 src/
3 cart.py
4 tests/
5 test_cart.py
6 agent_harness.py
7 pyproject.toml
```

### 7.2 初始业务代码：src/cart.py

代码块



```

1 from decimal import Decimal
2
3
4 def calculate_total(items, user_type="normal"):
5 """
6 items: [{"price": "60.00", "qty": 2}]
7 rules:
8 - subtotal >= 100: minus 10
9 - vip user: 5% off after threshold discount
10 - return Decimal with 2 decimal places
11 """
12 total = 0.0
13
14 for item in items:
15 total += float(item["price"]) * item["qty"]
16
17 if total >= 100:
18 total -= 10
19
20 if user_type == "vip":
21 total = total * 0.95
22
23 return Decimal(str(round(total, 2)))

```

这段代码能跑，但有问题：金额用了 `float`。在真实业务里，金额不要用 `float`，因为会有精度问题。

### 7.3 测试代码： `tests/test_cart.py`

代码块

```

1 from decimal import Decimal
2 import pytest
3
4 from src.cart import calculate_total
5
6
7 def test_normal_user_threshold_discount():
8 items = [{"price": "60.00", "qty": 2}]
9 assert calculate_total(items) == Decimal("110.00")
10
11
12 def test_vip_discount_after_threshold_discount():
13 items = [{"price": "60.00", "qty": 2}]
14 assert calculate_total(items, user_type="vip") == Decimal("104.50")
15
16
17 def test_no_float_precision_issue():
18 items = [
19 {"price": "0.10", "qty": 3},
20 {"price": "0.20", "qty": 1},
21]
22 assert calculate_total(items) == Decimal("0.50")
23
24
25 def test_negative_price_rejected():
26 items = [{"price": "-1.00", "qty": 1}]
27 with pytest.raises(ValueError):
28 calculate_total(items)

```

## 7.4 Harness 脚本: `agent_harness.py`

代码块

```
1 import json
2 import shutil
3 import subprocess
4 from datetime import datetime
5 from pathlib import Path
6
7
8 ROOT = Path(__file__).parent
9 TARGET_FILE = ROOT / "src" / "cart.py"
10 TRACE_FILE = ROOT / "trace.json"
11
12
13 PATCH_PROPOSAL = '''from decimal import Decimal, ROUND_HALF_UP
14
15
16 def money(value):
17 return Decimal(str(value)).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
18
19
20 def calculate_total(items, user_type="normal"):
21 """
22 items: [{"price": "60.00", "qty": 2}]
23 rules:
24 - subtotal >= 100: minus 10
25 - vip user: 5% off after threshold discount
26 - return Decimal with 2 decimal places
27 """
28 total = Decimal("0.00")
29
30 for item in items:
31 price = Decimal(str(item["price"]))
32 qty = int(item["qty"])
33
34 if price < 0:
35 raise ValueError("price must not be negative")
36
37 if qty < 0:
38 raise ValueError("qty must not be negative")
39
40 total += price * qty
41
42 if total >= Decimal("100.00"):
43 total -= Decimal("10.00")
44
45 if user_type == "vip":
46 total = total * Decimal("0.95")
47
48 return money(total)
49 '''
50
51
52 class Harness:
53 def __init__(self):
54 self.trace = {
55 "task": "fix money calculation without float",
56 "started_at": datetime.utcnow().isoformat() + "Z",
57 "steps": [],
58 }
```

```

59
60 def log(self, step_type, **kwargs):
61 record = {"type": step_type, **kwargs}
62 self.trace["steps"].append(record)
63 print(json.dumps(record, ensure_ascii=False))
64
65 def run(self, command):
66 self.log("run_command", command=command)
67 result = subprocess.run(
68 command,
69 cwd=ROOT,
70 shell=True,
71 text=True,
72 stdout=subprocess.PIPE,
73 stderr=subprocess.STDOUT,
74)
75 self.log(
76 "command_result",
77 exit_code=result.returncode,
78 output_tail=result.stdout[-1200:],
79)
80 return result.returncode == 0
81
82 def read_file(self, path):
83 self.log("read_file", path=str(path.relative_to(ROOT)))
84 return path.read_text(encoding="utf-8")
85
86 def write_file(self, path, content):
87 allowed = path.resolve() == TARGET_FILE.resolve()
88 if not allowed:
89 raise PermissionError(f"Not allowed to write {path}")
90
91 self.log("write_file", path=str(path.relative_to(ROOT)))
92 path.write_text(content, encoding="utf-8")
93
94 def save_trace(self, status):
95 self.trace["finished_at"] = datetime.utcnow().isoformat() + "Z"
96 self.trace["final_status"] = status
97 TRACE_FILE.write_text(
98 json.dumps(self.trace, ensure_ascii=False, indent=2),
99 encoding="utf-8",
100)
101
102 def execute(self):
103 backup = TARGET_FILE.with_suffix(".py.bak")
104 shutil.copyfile(TARGET_FILE, backup)
105
106 try:
107 self.read_file(TARGET_FILE)
108
109 baseline_ok = self.run("python -m pytest -q")
110 if baseline_ok:
111 self.log("baseline", message="tests already pass")
112 self.save_trace("already_passed")
113 return
114
115 self.write_file(TARGET_FILE, PATCH_PROPOSAL)
116
117 fixed_ok = self.run("python -m pytest -q")
118 if not fixed_ok:
119 shutil.copyfile(backup, TARGET_FILE)
120 self.log("rollback", reason="tests failed after patch")

```

```

121 self.save_trace("failed_and_rolled_back")
122 return
123
124 self.run("python -m py_compile src/cart.py")
125 self.log("done", message="patch accepted because tests passed")
126 self.save_trace("passed")
127
128 finally:
129 if backup.exists():
130 backup.unlink()
131
132
133 if __name__ == "__main__":
134 Harness().execute()

```

## 7.5 pyproject.toml

代码块

```

1 [tool.pytest.ini_options]
2 pythonpath = ["."]
3 testpaths = ["tests"]

```

## 7.6 运行方式

代码块

```

1 pip install pytest
2 python agent_harness.py

```

预期输出类似：

代码块

```

1 {"type": "read_file", "path": "src/cart.py"}
2 {"type": "run_command", "command": "python -m pytest -q"}
3 {"type": "command_result", "exit_code": 1, "output_tail": "..."}
4 {"type": "write_file", "path": "src/cart.py"}
5 {"type": "run_command", "command": "python -m pytest -q"}
6 {"type": "command_result", "exit_code": 0, "output_tail": "4 passed ..."}
7 {"type": "run_command", "command": "python -m py_compile src/cart.py"}
8 {"type": "done", "message": "patch accepted because tests passed"}

```

同时生成 `trace.json`，记录这次 Agent 修复过程。

## 7.7 这个例子体现了什么？

这个小例子很简单，但它已经有 Harness Engineering 的核心味道：

第一，Agent 不能随便改任何文件。脚本里限制它只能写 `src/cart.py`。

第二，修改不能靠嘴说完成，必须跑测试。

第三，失败要回滚。补丁如果跑不通，就恢复原文件。

第四，全过程有 trace。以后你可以复盘它读了什么、改了什么、跑了什么命令、为什么接受或拒绝补丁。

第五，完成标准是外部验证，不是模型自信。

这就是 Harness Engineering 最重要的思想：



让 AI 写代码不是难点，让 AI 写出来的代码被可靠验证，才是难点。

## 8. 把这个最小 Harness 扩展成真实工程系统

上面的脚本只是玩具版本。真实项目可以逐步加东西。

### 8.1 加入 AGENTS.md

代码块

```
1 # AGENTS.md
2
3 ## Project
4 This project handles cart pricing and discount calculation.
5
6 ## Rules
7 - Money must use Decimal.
8 - Do not use float for price calculation.
9 - Any pricing change must include tests.
10 - Do not change public function names unless requested.
11
12 ## Commands
13 - Run tests: python -m pytest -q
14 - Compile check: python -m py_compile src/cart.py
15
16 ## Done When
17 - All tests pass.
18 - No float is used for money.
19 - Edge cases are covered.
```

Agent 每次进入仓库先读这个文件。

### 8.2 加入静态扫描

你可以增加一个简单检查：如果业务代码里出现 `float()`，直接失败。

代码块

```
1 def check_no_float_for_money():
2 content = TARGET_FILE.read_text(encoding="utf-8")
3 forbidden = ["float("]
4 for token in forbidden:
5 if token in content:
6 raise AssertionError(f"Forbidden token found: {token}")
```

然后在 Harness 里跑：

代码块

```
1 check_no_float_for_money()
```

这就是 deterministic check。它不靠模型判断，规则明确。

### 8.3 加入 PR 输出

Agent 完成后，不只是说“好了”，而是生成：

```
1 ## Summary
2 - Replaced float-based money calculation with Decimal.
3 - Added validation for negative price and quantity.
4 - Preserved discount order: threshold discount first, VIP discount second.
5
6 ## Verification
7 - python -m pytest -q: passed
8 - python -m py_compile src/cart.py: passed
9
10 ## Risks
11 - Currency-specific rounding rules are not yet configurable.
```

这个输出可以进入 PR 描述。

## 8.4 加入 CI Gate

本地 Harness 只是第一关。进入团队后，还要接 CI：

代码块

```
1 name: pricing-check
2
3 on:
4 pull_request:
5 paths:
6 - "src/**"
7 - "tests/**"
8
9 jobs:
10 test:
11 runs-on: ubuntu-latest
12 steps:
13 - uses: actions/checkout@v4
14 - uses: actions/setup-python@v5
15 with:
16 python-version: "3.11"
17 - run: pip install pytest
18 - run: python -m pytest -q
```

Agent 可以开 PR，但 CI 不通过就不能合并。

## 8.5 加入人工审批点

比如：

代码块

```
1 如果改动涉及 src/payment/ 或 migrations/:
2 - Agent 可以提出 patch;
3 - Agent 可以跑测试;
4 - Agent 不允许自动合并;
5 - 必须由 human reviewer 审查。
```

这类规则比“相信 AI”靠谱得多。

## 9. Harness Agent 在 AI 应用开发里的典型用法

很多人以为 Harness Engineering 只适合代码生成。其实不是。只要你的 AI 应用需要 “多步执行、调用工具、依赖外部状态、需要验证结果”，就需要 Harness。

## 9.1 代码修复 Agent

输入：

代码块

- 1 线上出现错误：Decimal conversion syntax。
- 2 请定位原因、修复、补测试。

Harness 提供：

- 读取日志工具；
- 搜索代码工具；
- 修改文件工具；
- 测试工具；
- diff 输出；
- PR 创建工具。

完成标准：

- 新增复现测试；
- 修复后测试通过；
- 输出根因分析。

## 9.2 PR Review Agent

输入：

代码块

- 1 审查这个 PR 是否有安全、性能、架构问题。

Harness 提供：

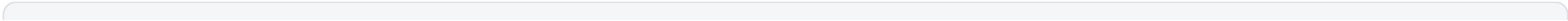
- git diff；
- changed files；
- 代码规范；
- 安全 checklist；
- 历史架构决策；
- 评论 API。

完成标准：

- 只评论具体文件和行；
- 不输出泛泛建议；
- 高风险问题必须说明影响和修复建议；
- 不阻塞低风险风格问题。

## 9.3 数据分析 Agent

输入：



代码块 分析 2026 年 Q1 用户留存下降原因。

Harness 提供：

- 查询只读数据仓库；
- 数据字典；
- 指标口径；
- Python notebook；
- 图表生成；
- 结果校验；
- 敏感字段脱敏。

完成标准：

- SQL 可复现；
- 指标口径引用明确；
- 图表有数据来源；
- 结论和证据对应；
- 不泄漏个人隐私数据。

9.4 运维排障 Agent

输入：

代码块

1 API 错误率从 1% 上升到 8%，帮我定位。

Harness 提供：

- logs；
- metrics；
- tracing；
- deploy history；
- incident runbook；
- rollback 工具；
- 权限审批。

完成标准：

- 找出时间线；
- 关联最近变更；
- 给出可能根因；
- 提供低风险缓解建议；
- 回滚必须人工确认。

9.5 文档维护 Agent

输入：

代码块



1 根据最新代码更新 API 文档。

Harness 提供：

- OpenAPI spec；
- 代码扫描；
- 文档目录；
- markdown lint；
- preview build。

完成标准：

- 文档和接口一致；
- 示例能运行；
- 链接不失效；
- PR 描述说明变更范围。

## 10. Harness.io Worker Agents 怎么理解？

如果你用的是 Harness.io 这个平台，它的 Worker Agents 可以看成一种面向软件交付流程的 Agent 运行机制。

它的典型特点包括：

- 在项目范围里创建和管理 Agent；
- 可以从 Marketplace 使用预置 Agent；
- 可以 fork 后定制；
- 可以写 Agent instructions；
- 可以配置 inputs、environment variables、MCP connectors；
- 可以让 Agent 进入 pipeline；
- 可以做 PR review、pipeline failure summarizer、IaC safety、code coverage、manifest remediation 等任务；
- 可以通过 Harness AI Chat、IDE 或 Harness MCP Server 创建和管理 Agent。

换句话说，它把 Agent 放在 CI/CD 和 DevOps 的上下文里，而不是让 Agent 只停留在聊天窗口里。

一个示意性的 Agent 配置可能长这样：

代码块

```
1 agent:
2 name: PR Reviewer Agent
3 description: Review pull requests for security, schema, and architecture risks.
4 instructions: |
5 You are a strict but practical PR reviewer.
6 Focus on:
7 - security risks
8 - breaking API changes
9 - database migration safety
10 - missing tests
11 Do not comment on minor style issues unless they affect readability.
12 inputs:
13 - name: pull_request_url
14 type: string
15 required: true
16 tools:
17 - type: mcp
```

```
18 name: github
19 - type: mcp
20 name: harness
21 output:
22 format: markdown
23 requirements:
24 - include risk level
25 - include file references
26 - include suggested fixes
```

注意，这只是帮助理解的示意，不是让你照抄到生产。真正使用时要以平台当前文档和你的组织权限模型为准。

它背后的思想和本文讲的一样：

Agent 不是孤零零的聊天机器人，而是嵌进工程流程里的工作单元。

## 11. MCP 和 Harness Engineering 的关系

MCP，全称 Model Context Protocol，可以理解成“AI 应用连接外部工具和数据源的一种标准接口”。

如果把 Agent 比作一个会思考的操作员，那么 MCP 就像一组标准插口。通过这些插口，Agent 可以连接：

- 文件系统；
- 数据库；
- GitHub；
- Jira；
- Slack；
- Google Drive；
- Figma；
- 内部 API；
- 搜索服务；
- 部署平台；
- 监控平台。

MCP 和 Harness Engineering 的关系是：

- MCP 解决“工具怎么标准化接入”的问题；
- Harness Engineering 解决“工具接入后怎么安全、可靠、可验证地使用”的问题。

也就是说，有 MCP 不等于有好 Harness。你可以给 Agent 接 100 个 MCP 工具，但如果没有权限、验证和任务边界，它仍然可能迷路。

好的 Harness 会对 MCP 工具做几件事：

第一，减少工具数量。不是所有工具都给 Agent，而是按任务给最少必要工具。

第二，给工具加说明。Agent 要知道这个工具适合什么场景，输入输出是什么。

第三，给工具加权限。读和写分开，高风险操作需要审批。

第四，记录工具调用。以后可以复盘 Agent 做了什么。

第五，给工具结果做结构化。不要让 Agent 在一大坨日志里猜重点。

## 12. 从“提示词文件”到“仓库知识系统”

很多团队一开始用 Agent，会写一个巨大的 `AGENTS.md` 或 `CLAUDE.md`。里面什么都有：

- 项目背景；
- 架构说明；
- 编码规范；
- API 说明；
- 数据库说明；
- 测试命令；
- 部署步骤；
- 常见坑；
- 产品规划；
- 历史债务；
- 甚至团队偏好。

这样做前期有用，但很快会变成垃圾堆。

更好的做法是：入口短，知识分层。

代码块

```
1 AGENTS.md
2 docs/
3 architecture.md
4 testing.md
5 coding-style.md
6 product/
7 pricing.md
8 checkout.md
9 decisions/
10 0001-money-decimal.md
11 0002-api-versioning.md
12 runbooks/
13 incident-api-error-rate.md
```

AGENTS.md 只告诉 Agent 去哪里找。

例如：

代码块

```
1 # AGENTS.md
2
3 ## Start Here
4 Read this file first. It is a map, not the full manual.
5
6 ## Architecture
7 See docs/architecture.md.
8
9 ## Pricing Rules
10 See docs/product/pricing.md.
11
12 ## Testing
13 See docs/testing.md.
14
15 ## Decisions
16 Before changing money calculation, read:
17 - docs/decisions/0001-money-decimal.md
18
19 ## Done Criteria
```

```
20 - tests pass
21 - lint passes
22 - docs updated if behavior changed
23 - PR summary includes verification result
```

这种结构有几个好处：

- 上下文更省；
- 文档更容易维护；
- Agent 更容易按需读取；
- 规则可以被测试；
- 过期内容更容易发现；
- 团队成员也能看懂。

### 13. Harness Engineering 和传统软件工程不是对立的

有些人会把 AI coding 说得像“传统软件工程要被淘汰”。这很危险。

现实更可能是：传统软件工程里的很多好习惯，会在 AI Agent 时代变得更重要。

比如：

- 清晰的模块边界；
- 自动化测试；
- 类型系统；
- CI/CD；
- code review；
- 文档；
- ADR；
- 可观测性；
- 权限控制；
- 灰度发布；
- 回滚机制。

以前这些东西是为了帮助人类团队协作。现在它们还多了一个作用：帮助 Agent 正确工作。

一个没有测试、没有文档、没有边界、没有 CI 的项目，人类维护起来已经困难；让 Agent 维护，只会更乱。

所以 Harness Engineering 不是“反工程”，而是“把工程纪律变成 Agent 的工作环境”。

### 14. Agent-first 开发里，工程师的角色怎么变？

在 Vibe Coding 阶段，工程师像一个“AI 指挥员”：

```
代码块
1 写这个。
2 改那里。
3 报错了，修一下。
4 再美化一下。
```

在 Harness Engineering 阶段，工程师更像一个“系统设计者”和“质量负责人”。

你要关心：

- 需求如何表达成规格；
- 项目知识如何组织；
- Agent 能用哪些工具；
- 哪些操作需要审批；
- 怎样验证完成；
- 怎样记录过程；
- 怎样评估质量；
- 怎样让失败经验沉淀到系统里。

也就是说，工程师不一定逐行写代码，但要对系统结果负责。

未来很可能出现一种新型工程能力：

不只是会写代码，而是会设计一个让 Agent 稳定产出好代码的环境。

这就是 Harness Engineering 的真正价值。

## 15. 一个真实团队可以怎么落地？

如果你现在带一个小团队，想从 Vibe Coding 走向 Harness Engineering，不建议一上来搞很重的平台。可以分四步走。

### 第一步：补最小项目上下文

先加三个文件：

代码块

- 1 AGENTS.md
- 2 docs/architecture.md
- 3 docs/testing.md

AGENTS.md 写：

代码块

- ```
1  # AGENTS.md
2
3  ## Project
4  一句话说明项目。
5
6  ## Read Before Coding
7  - docs/architecture.md
8  - docs/testing.md
9
10 ## Commands
11 - install:
12 - test:
13 - lint:
14 - typecheck:
15
16 ## Rules
17 - 不要改哪些目录
18 - 关键业务规则
19 - 代码风格
20
21 ## Done When
```


- 22 - 测试通过
- 23 - lint 通过
- 24 - 输出变更说明

别写太长。先让 Agent 不迷路。

第二步：把“完成标准” 机器化

不要只写：

代码块

- 1 代码质量要好。

要写成可执行命令：

代码块

- 1 python -m pytest -q
- 2 npm run lint
- 3 npm run typecheck
- 4 npm run build

能机器判断，就不要靠感觉判断。

第三步：把 Agent 变更统一走 PR

不要让 Agent 直接改主分支。推荐：

- 每个任务一个 branch；
- Agent 提交 diff；
- CI 跑；
- 人类 review；
- 合并。

PR 模板可以这样写：

代码块

- 1 ## What changed?
- 2
- 3 ## Why?
- 4
- 5 ## Verification
- 6 - [] Tests passed
- 7 - [] Lint passed
- 8 - [] Typecheck passed
- 9 - [] Manual check completed
- 10
- 11 ## Risk
- 12 Low / Medium / High
- 13
- 14 ## Rollback
- 15 How to rollback this change?

第四步：记录失败并改进 Harness

每次 Agent 出错，不要只骂模型。要问：

- 是任务没写清楚？
- 是上下文缺了？
- 是工具太多？
- 是没有测试？
- 是权限太大？
- 是日志太乱？
- 是完成标准不明确？
- 是旧文档误导了它？

然后把经验沉淀进：

- AGENTS.md ；
- docs/testing.md ；
- 新测试；
- 新 lint rule；
- 新审批规则；
- 新 runbook；
- 新 eval case。

这就是闭环。

16. 如何评价一个 Harness 好不好？

可以用下面这张表。

维度	差的 Harness	好的 Harness
任务输入	只有一句自然语言	有目标、非目标、边界、完成标准
上下文	全塞进一个长提示词	短入口 + 分层文档
工具	工具很多但语义混乱	少量稳定、可组合工具
权限	什么都能做	最小权限 + 高风险审批
验证	AI 自称完成	测试、lint、build、review
状态	靠聊天记录	计划文件、状态文件、trace
失败处理	失败就重问	定位原因、回滚、沉淀规则
团队协作	个人玩具	PR、CI、文档、审查
可改进性	每次从零开始	trace + eval + feedback loop

如果你的团队现在还在“复制 AI 代码到项目里”，那不一定错，但它还在早期阶段。

如果你的团队已经开始让 Agent 读文档、跑测试、开 PR、留 trace、按规则工作，那你已经在走向 Harness Engineering。

17. 常见误区

17.1 误区一：提示词越长越好

不对。提示词太长会稀释重点。

更好的方式是：

- 入口短；
- 文档分层；
- 规则明确；
- 让 Agent 按需读取。

17.2 误区二：工具越多越强

不对。工具多会增加选择成本。Agent 可能不知道该用哪个，还可能把低风险任务搞成高风险操作。

工具要少、稳、可组合。

17.3 误区三：AI 通过一次测试就能放心

不一定。测试只能证明你测过的地方没问题。关键业务还需要：

- 边界测试；
- 回归测试；
- 人类 review；
- 灰度；
- 监控；
- 回滚。

17.4 误区四：Agent 会自动理解团队品味

不会。团队的架构偏好、命名习惯、抽象边界、错误处理方式，都要写下来，最好还能用 lint、测试或 review checklist 固化。

17.5 误区五：Harness Engineering 只适合大公司

不对。小团队更需要它，因为小团队没有那么多肉 review。哪怕只加一个 `AGENTS.md` 和测试命令，也比纯 Vibe Coding 稳定很多。

18. 一个适合小团队的 Harness 模板

你可以直接从下面这个结构开始：

代码块

```
1  my-project/
2    AGENTS.md
3    docs/
4      architecture.md
5      testing.md
6      coding-style.md
7      decisions/
8        0001-key-decision.md
9    scripts/
10     verify.sh
11  .github/
12    pull_request_template.md
```

scripts/verify.sh

代码块

```
1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  echo "Running tests..."
5  python -m pytest -q
6
7  echo "Running compile check..."
8  python -m compileall src
9
10 echo "Verification passed."
```

AGENTS.md

代码块

```
1  # AGENTS.md
2
3  ## Project
4  This is a small Python service.
5
6  ## Before You Start
7  Read:
8  - docs/architecture.md
9  - docs/testing.md
10
11 ## Commands
12 Use this command before claiming completion:
13
14 ```bash
15 bash scripts/verify.sh
```

Rules

- Keep changes minimal.
- Add tests for behavior changes.
- Do not introduce network calls unless requested.
- Do not edit secrets, credentials, or deployment configs.
- Do not claim completion if verification fails.

Response Format

When done, report:

1. Summary
2. Files changed
3. Verification result
4. Risks
5. Follow-up suggestions

代码块

```
1
```

```
2   ### `docs/testing.md`
3
4   ```markdown
5   # Testing
6
7   ## Unit Tests
8   Run:
9
10  ```bash
11  python -m pytest -q
```

Required Coverage

For business logic changes, include:

- normal case
- boundary case
- invalid input
- regression case if fixing a bug

Completion Rule

A task is not done until tests pass.

代码块

```
1
2   ### PR 模板
3
4   ```markdown
5   ## Summary
6
7   ## Files Changed
8
9   ## Verification
10  - [ ] bash scripts/verify.sh
11
12  ## Risk Level
13  Low / Medium / High
14
15  ## Notes for Reviewer
```

这个模板很简单，但已经能让 Agent 的表现稳定不少。

19. Harness Engineering 的更高级方向：Evals、Trace 和自动改进

当团队开始大量使用 Agent 后，你会遇到一个新问题：

我们怎么知道 Agent 越来越好了，还是只是感觉越来越好了？

这时候要引入 evals。

一个 eval case 可以长这样：

代码块

```
1   id: pricing-negative-price
2   task: Fix cart pricing to reject negative prices.
3   repo_state: fixture/pricing_bug
```



```
4   expected:
5   tests:
6     - tests/test_cart.py::test_negative_price_rejected
7   forbidden_patterns:
8     - "float("
9   required_files_changed:
10    - "src/cart.py"
```

每次改 Harness、改提示词、改工具、改模型，都跑一批 evals，看成功率、耗时、token、失败类型有没有变化。

Trace 的作用是告诉你失败为什么发生。

例如：

代码块

```
1  {
2    "case_id": "pricing-negative-price",
3    "result": "failed",
4    "failure_type": "missing_test",
5    "notes": "Agent fixed implementation but did not add regression test."
6  }
```

你看到这种失败，就不应该只说“模型不行”。你可以改 Harness：

- 在 AGENTS.md 里加“修 bug 必须先写失败测试”；
- 在 verify 脚本里检查测试文件是否变更；
- 在 PR 模板里强制填写 regression test；
- 在 Agent 输出里要求列出新增测试。

这就是 Harness 自我改进。

20. 从工程角度重新理解“AI 应用开发”

过去很多 AI 应用开发教程会重点讲：

- 怎么调 API；
- 怎么写 prompt；
- 怎么做 RAG；
- 怎么接向量数据库；
- 怎么做 function calling；
- 怎么做多 Agent。

这些当然重要。但如果你真的要做一个能上线、能维护、能团队协作的 AI 应用，还要问更工程化的问题：

20.1 这个 Agent 的输入契约是什么？

用户能说任何话，但系统不能接受任何含糊目标直接执行。你要把用户输入转成任务规格。

20.2 这个 Agent 能用哪些工具？

工具不是越多越好。每个工具都要有：

- 名称；
- 描述；
- 输入 schema；

- 输出 schema；
- 权限；
- 风险级别；
- 是否需要审批。

20.3 这个 Agent 的状态放在哪里？

短期状态可以放对话里，长期状态最好放数据库、文件、任务系统或 trace store。

20.4 这个 Agent 怎么判断完成？

不能只靠模型说完成。要有可验证标准。

20.5 这个 Agent 怎么失败？

失败也要设计。比如：

- 工具失败；
- 权限不足；
- 上下文不足；
- 测试失败；
- 外部 API 超时；
- 用户目标冲突；
- 安全策略拒绝。

每种失败都应该有清晰输出，而不是“抱歉我失败了”。

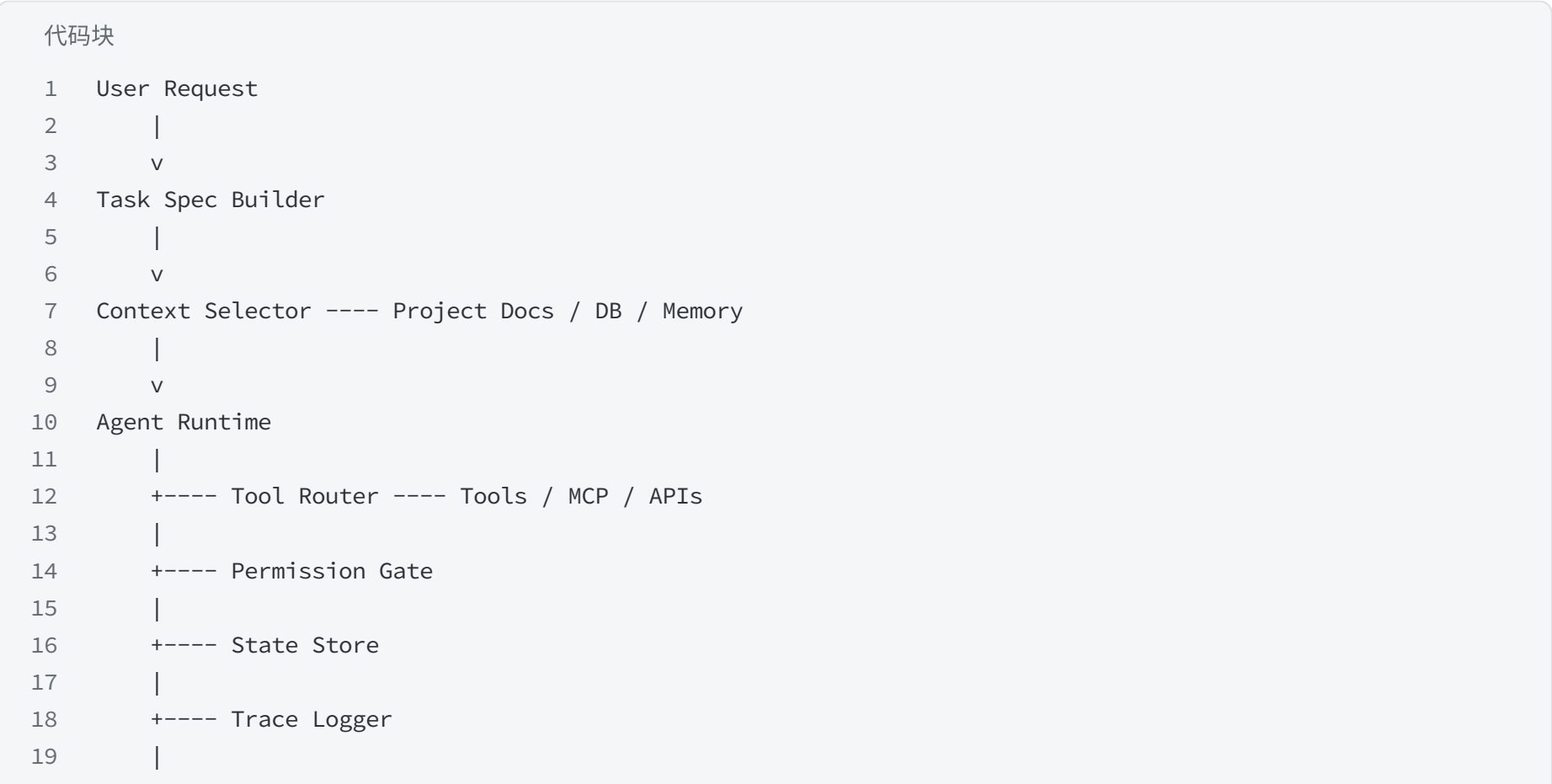
20.6 这个 Agent 怎么被审计？

如果 Agent 能改代码、查数据、发消息、开 PR、触发部署，那就必须能审计。

审计不是大公司才需要，小团队也需要。否则出了问题没人知道它做过什么。

21. 一个 AI 应用里的 Harness 架构图

可以把系统想成这样：



```
20      v
21  Verification Layer ----- Tests / Evals / Rules / Human Review
22      |
23      v
24  Final Output / PR / Report / Action
```

每一层都有作用。

- Task Spec Builder：把自然语言变成明确任务。
- Context Selector：挑相关上下文，不乱塞。
- Agent Runtime：让模型规划和执行。
- Tool Router：管理工具调用。
- Permission Gate：控制风险。
- State Store：保存进度。
- Trace Logger：记录过程。
- Verification Layer：判断是否完成。
- Final Output：输出给用户或进入工程流程。

如果你只调用一次 LLM API，那不是 Harness。那只是模型调用。

如果你有工具、有状态、有验证、有权限、有 trace，那才开始像一个 Harness。

22. Harness Engineering 对产品经理、小白和非技术人的意义

这件事不只是程序员关心。

非技术人使用 AI 做产品原型时，也会遇到同样问题。

你用 Vibe Coding 做一个产品 Demo，很容易说：

```
代码块
1  帮我做一个 CRM 系统。
```

AI 可能很快给你页面。但你真正需要的不是“看起来像 CRM”，而是：

- 客户字段有哪些；
- 谁可以看客户；
- 销售阶段怎么流转；
- 数据怎么导入导出；
- 权限怎么做；
- 删除客户是否可恢复；
- 操作日志怎么留；
- 移动端要不要适配；
- 报表口径是什么；
- 哪些功能第一版不做。

这就是任务规格。

哪怕你不会写代码，也可以做 Harness Engineering 的一部分：把目标、规则、验收标准说清楚。

例如你可以这样要求 AI：

```
代码块
1 你先不要写代码。请先帮我整理一份产品规格，包含：
2
3   1. 用户角色
4   2. 核心流程
5   3. 数据字段
6   4. 页面列表
7   5. 第一版做什么
8   6. 第一版不做什么
9   7. 验收标准
10  8. 风险和疑问
11
12 等我确认后，再开始生成代码。
```

这比直接“帮我写系统”靠谱得多。

23. 实践清单：以后怎么和 Coding Agent 合作

23.1 开始任务前

你应该准备：

```
代码块
1  ## Goal
2  我要做什么？
3
4  ## Why
5  为什么做？
6
7  ## Scope
8  这次包括什么？
9
10 ## Non-goals
11 这次不包括什么？
12
13 ## Constraints
14 有什么技术限制、业务限制、安全限制？
15
16 ## Done When
17 怎样才算完成？
18
19 ## Verification
20 用什么命令或方法验证？
```

23.2 让 Agent 工作时

不要只说：

```
代码块
1 继续。
```

要说：

```
代码块
1 先读取 AGENTS.md 和 docs/testing.md。
2 然后制定计划。
```

- 3 计划确认后再改代码。
- 4 每完成一个 milestone 跑一次测试。
- 5 如果测试失败，先解释失败原因，再修复。

23.3 Agent 完成后

不要只看它的总结。你要看：

- diff;
- tests;
- lint;
- build;
- edge cases;
- 是否改了不该改的文件；
- 是否新增了不必要的依赖；
- 是否把问题复杂化；
- 是否修改了测试来迎合错误实现；
- 是否留下了说明文档。

23.4 任务结束后

把经验沉淀下来：

- 这次哪里说得不清楚？
- 哪个测试缺失？
- 哪条规则应该写进 AGENTS.md？
- 哪个工具权限太大？
- 哪个文档过期？
- 哪类任务适合 Agent？
- 哪类任务必须人类主导？

24. 未来会发生什么？

我认为未来 AI 应用开发会出现几个变化。

24.1 “写代码”会继续变便宜

普通功能、样板代码、测试补齐、文档更新、迁移脚本，会越来越多地由 Agent 完成。

但这不代表工程师不重要。因为真正难的是判断什么该做、怎么做才稳、风险在哪里、系统长期怎么维护。

24.2 仓库会变得更 “Agent-readable”

未来的好仓库，不只是人能看懂，也要 Agent 能看懂。

它会有：

- 清晰文档；
- 机器可执行验证；
- 结构化任务；
- 明确模块边界；
- 标准化工具入口；

- 可追踪决策记录。

24.3 CI/CD 会变成 Agent 的工作场所

Agent 不会只在 IDE 里。它会进入：

- issue triage；
- PR review；
- CI failure repair；
- release note；
- incident response；
- dependency upgrade；
- security remediation；
- test generation；
- migration planning。

软件交付流水线会越来越像“人类 + Agent”的协作系统。

24.4 Harness 会成为 AI 原生应用的核心资产

模型会更新，工具会变化，框架会流行又过时。但你团队沉淀下来的 Harness 会变成资产：

- 任务模板；
- 测试体系；
- 文档体系；
- 工具边界；
- trace 数据；
- eval 集合；
- review 规则；
- 权限模型；
- 失败案例库。

这些东西越积越多，Agent 的工作质量才会越来越稳。

25. 总结：从“让 AI 写”到“让 AI 在系统里正确地写”

Vibe Coding 让我们第一次感受到：用自然语言驱动软件开发是可能的。它降低了门槛，提高了速度，也让很多想法能快速变成原型。

但如果我们停在 Vibe Coding，就会遇到代码质量、上下文漂移、验证不足、安全边界、长期维护等问题。

Harness Engineering 解决的是下一阶段的问题：

- 不只是 prompt，而是任务规格；
- 不只是上下文，而是项目知识系统；
- 不只是工具调用，而是受控工具环境；
- 不只是生成代码，而是验证代码；
- 不只是一次运行，而是可追踪、可回滚、可复盘；
- 不只是人盯着 AI，而是人设计 AI 的工作系统。

｜ Vibe Coding 让 AI 动起来，Harness Engineering 让 AI 靠谱起来。

对个人开发者来说，你可以从一个 `AGENTS.md`、一个 `scripts/verify.sh`、一套测试开始。

对团队来说，你可以从 PR 流程、CI gate、Agent trace、任务模板开始。

对 AI 应用开发者来说，你要从一开始就设计：这个 Agent 能做什么、不能做什么、怎么知道做对了、失败后怎么处理、过程怎么审计。

未来真正拉开差距的，不只是“谁会用 AI 写代码”，而是：

谁能搭出一个让 AI 稳定交付的工程系统。